

MSc Research Progress Report

Coordinating Speculative Agent Workloads Over Data Backends

Middleware coordination policies for redundancy elimination
in multi-agent BIRD text-to-SQL workloads

Prepared: 13 June 2026
Author: gorenkinp@gmail.com
Programme: MSc Computer Science

Phase	Status	Key Deliverable
Phase 0 — Scaffold	✓ Complete	Repo, CI, BIRD data, configs
Phase 1a — Gold SQL / EX metric	✓ Complete	Execution accuracy harness
Phase 1b — JSONL traces	✓ Complete	Per-run trace schema
Phase 1c — Single-agent loop	✓ Complete	Tool-calling agent (3 models)
Phase 2 — Parallel agents (P0)	✓ Complete	N-replica coordinator + metrics
Phase 2a — Baseline redundancy	✓ Complete	Chapter 2 draft + plots
Early-stop optimisation	✓ Complete	Cost reduction analysis
Phase P1–P4 Middleware	Planned	Shared cache, sub-expr dedup

1. Project Overview

This MSc research investigates **middleware coordination policies** for multi-agent LLM systems that perform text-to-SQL translation over relational databases. The central hypothesis is that independent parallel agents waste substantial work through redundant database exploration, and that lightweight middleware — shared caches, early stopping, sub-expression propagation — can eliminate this waste while preserving or improving answer quality.

The benchmark is **BIRD** (Benchmark for Intelligent Reasoning over Diverse databases), specifically the mini-dev split (500 tasks, 11 SQLite databases). The evaluation metric is **Execution Accuracy (EX)**: the fraction of tasks where the agent's submitted SQL returns a result set matching the gold answer.

Research questions

- How much redundant database exploration occurs when N identical agents solve the same task independently (baseline P0)?
- Does early stopping (cancel sibling replicas when one finds a correct answer) reduce token cost without harming EX?
- Can shared SQL result caches (P1) eliminate duplicate explore queries, and by how much?
- Do richer coordination policies (P2–P4: sub-expression sharing, majority vote, ensemble) further reduce cost or improve accuracy?

Models under evaluation

Registry Key	Provider	API Model ID	Role
gpt-4o-mini	OpenAI	gpt-4o-mini	Primary baseline
gemini-2.5-flash	Google	gemini-2.5-flash	Evaluation matrix
deepseek-v3.2	DeepSeek	deepseek-chat (V3.2)	Evaluation matrix

2. System Architecture

The codebase follows a layered architecture. The innermost layer is a thin **SQLite execution harness** that matches the official BIRD evaluation logic. Above it sits an **LLM client** with provider-agnostic retry/backoff supporting OpenAI, Google Gemini, and DeepSeek via their respective SDKs. The **agent loop** implements a tool-calling cycle with two tools: `execute_sql` (read-only exploration) and `submit_sql` (final answer). A **coordination layer** spawns N replicas in a thread pool and applies a selection policy once all (or enough) replicas complete.

Key source modules

Module	Path	Responsibility
LLM client	src/llm/client.py	Provider-agnostic chat completions
Retry/backoff	src/llm/retry.py	Exponential backoff, rate-limit handling
Agent loop	src/agent/loop.py	Tool-calling loop, turn tracking, traces
Coordination	src/coord/parallel.py	Thread pool, early-stop logic
Policies	src/coord/policies.py	best_of_n, first_success, majority_vote
Redundancy analysis	src/coord/redundancy.py	String + AST dedup, sub-expr overlap

Module	Path	Responsibility
Baseline plots	src/coord/baseline_plots.py	Matplotlib scaling-curve figures
Config	src/config.py	Pydantic settings, YAML loading
BIRD tasks	src/bird/tasks.py	Task loader, SQLite path resolution
SQLite exec	src/db/sqlite_exec.py	Timeout-guarded execution

JSONL trace schema

Every agent run writes a structured JSONL trace to runs/<run_id>.jsonl. Coordinator runs additionally write runs/coord_<id>.jsonl. Traces record all events needed for offline redundancy analysis:

Event	Key fields
run_start	question_id, db_id, policy, seed, bird_split, agent_id
sql_execute	sql_raw, sql_role, latency_ms, row_count, result_sample, error
run_end	ex_correct, predicted_sql, gold_sql, wall_clock_ms
parallel_start	replicas, policy, early_stop flag
coordination_end	selected_agent_id, redundancy metrics, token totals

3. Phase-by-Phase Progress

Phase 0 — Project Scaffold

Complete

Repository structure, CI configuration (pytest), uv dependency management, BIRD mini-dev download scripts, YAML configuration system (Pydantic), model registry (configs/models.yaml), and environment setup verified across all three LLM providers.

Phase 1a — Gold SQL & Execution Accuracy

Complete

SQLite execution harness matching official BIRD evaluation.py logic. Gold-vs-gold sanity check (20/20 EX=1 confirmed). Scripts: run_one_gold.py, run_gold_sanity.py.

Phase 1b — JSONL Traces

Complete

Structured event logging for every agent run. Schema covers run lifecycle, all SQL executions (with latency and row counts), and final evaluation. Offline inspection via scripts/inspect_trace.py.

Phase 1c — Single-Agent Loop

Complete

Tool-calling agent with execute_sql / submit_sql tools. Validated on all three models. Batch runner with CSV/JSON output (run_batch.py). LLM retry/backoff layer added (exponential, jitter, per-provider rate-limit detection).

Phase 2 — Parallel Agent Coordinator

Complete

Thread-pool N-replica launcher with three selection policies: **best_of_n** (fewest-turn correct replica), **first_success** (first EX=1 replica), **majority_vote** (result-set bucket voting). Per-replica and coordinator JSONL traces. Redundancy metrics computed inline (string + AST dedup, token overhead ratio).

Phase 2a — Baseline Redundancy Analysis (P0)**Complete**

Replica-count sweep ($N \in \{3, 10, 25\}$) across all three models on 50-task smoke subset. Full redundancy report suite: explore duplication, sub-expression overlap, token overhead, wall-clock. Six scaling-curve figures (Matplotlib). Chapter 2 thesis draft auto-generated (scripts/generate_chapter2_draft.py).

Early-Stop Optimisation**Complete**

Early stopping (cancel sibling replicas when one achieves EX=1) evaluated apples-to-apples vs P0 at N=10. Key finding: DeepSeek saves **14.3% tokens** with no EX loss; Gemini saves 2.5%; GPT-4o mini shows marginal increase due to pre-emption timing.

Eval Matrix (all models × variations)**Complete**

Concurrent matrix runner (run_eval_matrix.py) executing single-agent and parallel variations across all three models simultaneously. Manifest JSON aggregates all batch results.

4. Key Experimental Results

4.1 Baseline Redundancy (P0)

The P0 baseline runs N independent replicas with no shared state. Results below are on the 50-task BIRD mini-dev smoke subset with best_of_n coordination policy.

Replicas	DeepSeek EX%	DeepSeek Redund%	Gemini EX%	Gemini Redund%	GPT-4o mini EX%	GPT-4o mini Redund%
3	60.0	54.1%	72.0	45.9%	58.0	50.9%
10	60.0	71.3%	74.0	73.6%	58.0	78.5%
25	64.0	82.9%	70.0†	81.5%	36.0†	90.2%

† API failures at N=25: Gemini 3 tasks, GPT-4o mini 14 tasks. EX on completed tasks: Gemini 74.5%, GPT-4o mini 50.0%.

Token overhead (N=3 vs N=25):

Model	Tokens at N=3	Tokens at N=25	Growth	Overhead at N=25
DeepSeek V3.2	2,193,564	18,889,223	8.6×	32.66×
Gemini 2.5 Flash	463,742	3,685,459	7.9×	26.55×
GPT-4o mini	732,572	4,026,175	5.5×	25.97×

4.2 Early-Stop vs P0 (N=10)

Early stopping cancels remaining replicas as soon as one achieves EX=1. Evaluated at N=10 with best_of_n policy (identical to P0 baseline).

Model	P0 EX%	ES EX%	P0 Redund%	ES Redund%	P0 Tokens	ES Tokens	Token Δ	ES Triggered
GPT-4o mini	58.0	60.0	78.5%	77.8%	2,400,807	2,432,022	+1.3%	30/50
Gemini 2.5 Flash	74.0	74.0	73.6%	73.6%	1,568,278	1,529,204	−2.5%	37/50
DeepSeek V3.2	60.0	60.0	71.3%	75.0%	7,792,210	6,676,709	−14.3%	30/50

DeepSeek V3.2 benefits most (−14.3% tokens, 30/50 tasks triggered, avg 5.1 replicas cancelled). GPT-4o mini shows marginal increase due to pre-emption timing at N=10.

5. Scaling-Curve Figures

All figures below are generated from the P0 baseline sweep ($N \in \{3, 10, 25\}$), three models, 50-task BIRD mini-dev smoke subset).

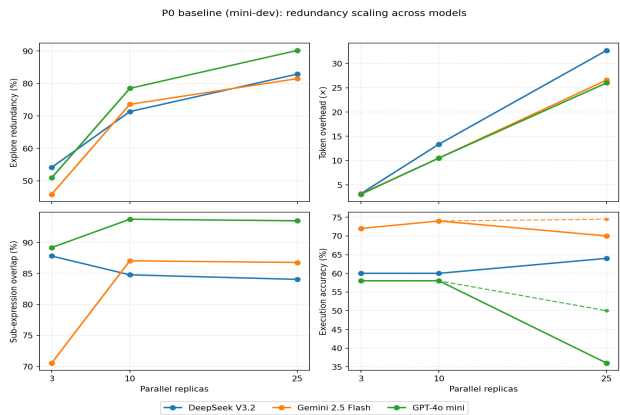


Figure 1. P0 baseline overview: four-panel summary across all models and replica counts.

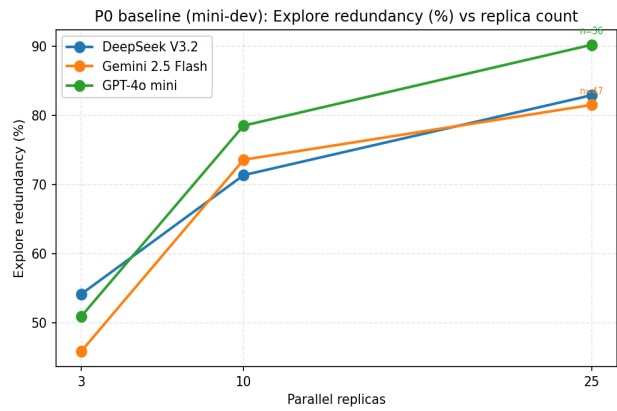


Figure 2. Mean explore-query redundancy vs replica count. Redundancy exceeds 80% at $N=25$ for all models.

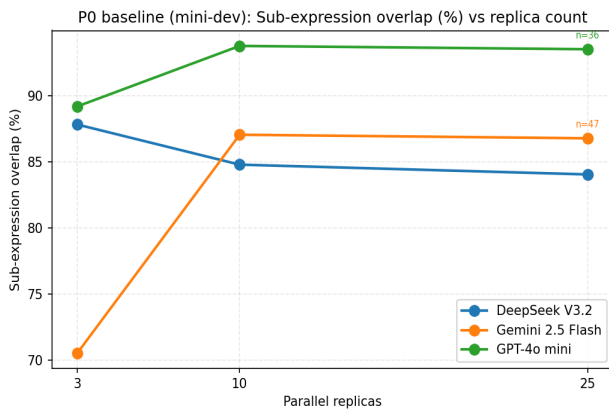


Figure 3. Sub-expression overlap: shared tables/columns/predicates across replicas. Reaches 84–94% at $N=3$ already.

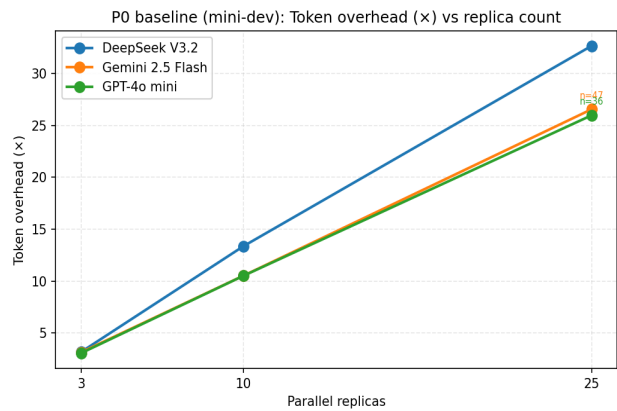


Figure 4. Token overhead ratio (total tokens / cheapest correct replica). Approaches 26–33 $\times$ at $N=25$.

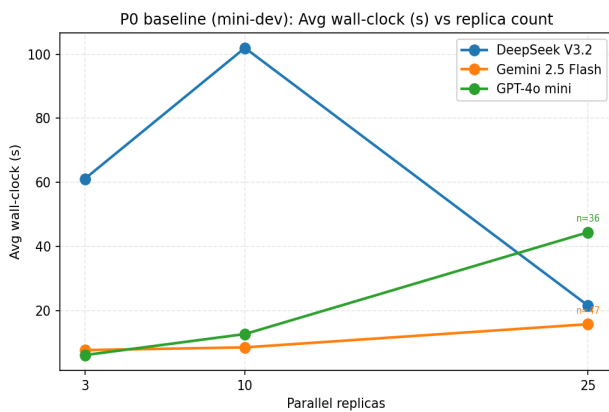


Figure 5. Wall-clock time vs replica count. Sub-linear scaling due to concurrency; DeepSeek higher due to longer per-replica latency.

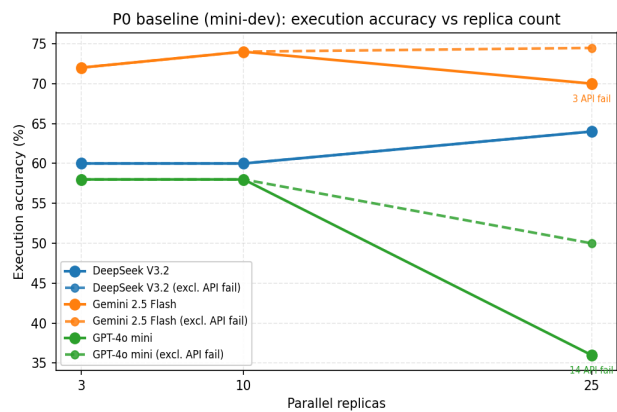


Figure 6. Execution accuracy vs replica count. Dashed lines exclude API-failure tasks at $N=25$.

6. Key Findings

Finding 1: Redundancy is the dominant cost of parallel agents

At N=10, roughly four out of five explore queries are duplicates. Token spend scales linearly with N (~10× at N=10, ~25–33× at N=25) while EX% moves only a few points. Parallelism buys reliability at a large and measurable token premium.

Finding 2: Unique exploration saturates quickly

The count of distinct explore SQL strings plateaus far below total explore volume. For Gemini, unique queries increase only 16% from N=3 to N=25 while total volume grows 7.9×. Replicas are not discovering more of the search space — they are re-executing the same probes.

Finding 3: Sub-expression overlap motivates fragment-level middleware

String-level deduplication (P1) handles exact copies; the 84–94% sub-expression overlap at N=3 already shows that replicas share tables, columns, and predicates even when full SQL strings differ. This motivates P2 (fragment-level sharing).

Finding 4: Early stopping saves tokens without harming accuracy

At N=10, DeepSeek V3.2 saves 14.3% of total tokens with zero EX loss. Gemini saves 2.5%. Early stopping is a simple, safe optimisation that should always be enabled in production-like deployments.

Finding 5: API reliability degrades at high N

GPT-4o mini at N=25 saw 14/50 tasks fail at the API layer (all retries exhausted), dropping headline EX from 50% (completed tasks only) to 36% (all tasks). Any high-N deployment must account for replica failure modes.

7. Planned Next Steps

The remaining thesis phases build the middleware stack on top of the P0 baseline established in Phases 0–2a:

P1 — Shared SQL Result Cache

A server-side LRU cache keyed by AST-normalised SQL. When a replica submits an explore query already executed by another replica, return the cached result set immediately without a database round-trip. Expected to eliminate the ~50–90% of duplicate string-identical queries measured in P0.

P2 — Sub-Expression Propagation

Extend P1 to share individual SQL fragments (table probes, predicate evaluations, join patterns) across replicas even when full SQL strings differ. Targets the 85–94% sub-expression overlap observed at N=3.

P3/P4 — Richer Coordination Policies

Evaluate majority_vote and cross-model ensemble coordination. Investigate whether sharing intermediate results (not just caching) changes the exploration frontier and improves EX%.

Scale-up to Full BIRD Dev

Re-run the baseline sweep and all middleware policies on the full 1,534-task BIRD dev set to validate that smoke-subset magnitudes generalise.

Thesis Write-up

Chapters 3–5 covering P1–P4 middleware evaluation; Chapter 6 discussion and conclusions.

8. Repository Artefacts

All code, data, traces, and reports are versioned in the project repository. Key artefacts produced to date:

Artefact	Location	Description
Source modules	src/	Agent loop, LLM client, coord policies, BIRD loader
Test suite	tests/	pytest unit tests (~15 test files)
Scripts	scripts/	18 runnable scripts covering all eval phases
Configs	configs/	YAML config, model registry, subset files
BIRD mini-dev	data/bird/mini_dev/	500-task dev set, 11 SQLite databases
Agent traces	runs/*.jsonl	Per-run JSONL traces (hundreds of files)
Batch CSVs/JSONs	runs/batches/	28+ batch result files across models
Baseline reports	runs/reports/	10 Markdown + JSON analysis reports
Scaling plots	runs/reports/plots/	6 publication-quality PNG figures
Chapter 2 draft	thesis/chapter2_baseline_redundancy.md	Thesis-ready write-up with embedded figures
Progress report	progress_report.pdf	This document

Git history: 2 commits to date. Branch: main. Working tree has active modifications across all core modules (retry logic, batch pacing, Gemini 2.5 migration complete).